GitHub WorkFlow

Repository Structure

The TIVIDY project is hosted in a single GitHub repository that serves as both the development workspace and the evidence of the team's engineering process. The repository is organised so that code, tests, documentation and design artefacts live together and are reviewed through the same pipeline.

```
tividy/
|-- README.md
|-- CONTRIBUTING.md
|-- Makefile                    build, test, run, valgrind, doxygen, docker targets
|-- Dockerfile                  Ubuntu 22.04 + GCC 11 + GoogleTest 1.12.1
|-- Doxyfile
|-- app/                        main.cpp, the TUI entry point
|-- include/                    public headers
|   |-- engine/                 core, events, connectors, items, policies,
|   |                           commands, definition, runtime, facade
|   |-- industries/             banking, telecom, demo
|   \-- ui/
|-- src/                        implementations (same layout as include/)
|-- tests/                      GoogleTest files, one binary each
|-- report/                     final PDF report and per-deliverable PDFs
|-- docs/
|   |-- vpp/                    Visual Paradigm projects
|   |-- img/final/              final diagram SVGs
|   |-- img/pa6/                Practical 6 diagrams (archived)
|   |-- archive/                Practical 6 submission
|   |-- research/               initial research and bibliography
|   |-- assets/
|   |-- misc/
|   \-- TIVIDY_Implementation_Plan.pdf
|-- .devcontainer/              enables Docker to run in Codespaces
\-- .github/                    CI workflows, PR template, labeler
```

The docs/ directory holds the research findings and references, diagrams and our PA6 document before beginning the implementation of TIVIDY. The repository records reflect the final artefact and the reasoning and processing that produced it.

Branching Strategy

main is protected. No member may push to it directly. Every change reaches main through a pull request opened from a branch whose name follows a fixed convention:

prefix/short-description

Where the allowed prefixes are feature/, fix/, docs/, refactor/, test/ and chore. The description may contain only letters, numbers, ".", "_" and "-". Examples include feature/work-item-state, docs/ClassDiagramV2.

The prefix is not cosmetic, it determines the label applied to the pull request automatically, and the CI (continuous integration) workflow that validates branch names will reject a branch that does not follow the convention. This keeps the branch list, the PR list and the label list consistent without manual bookkeeping. If there was any notably close collaboration between members or files, an integration branch was used.

Commit and Pull-Request Conventions

Commit messages and pull-request titles both follow the form:

type: what changed

Where the allowed types are feat, fix, docs, refactor, test and chore. feat corresponds to the feature/ branch prefix, the other types match their branch prefixes directly. E.g. docs: update class diagram with Visitor

Pull-Request Process

Every branch is opened as a pull request against main. The repository has a pull-request template that prompts the author to record:

- A one to three sentence summary of the change;
- The type of change;
- The design pattern(s) and classes the change affects
- How the change was tested;
- A checklist covering branch freshness, compilation, the avoidance of large if/else or switch chains, diagram updates, design-decision logging and a self-review of the diff

The template is the mechanism by which the project rules are checked at review time rather than remembered. A reviewer who sees an unchecked box or a missing pattern reference can ask for it directly and the author has already been prompted to supply it.

A pull request is merged only after:

1. Four approvals from teammates other than the author (meaning a total of five people have seen the change and believe it should be merged);
2. All review conversations resolved;
3. All automated checks passing;
4. The branch being up to date with main

Because approvals are dismissed when new commits are pushed, the reviewer effectively re-approves the state of the branch that is actually merged. This prevents a pull request from being approved in one form and merged in another.

Automated Checks

Four GitHub Actions workflows run on every pull request into main to maintain pull request hygiene:

- build: runs make on ubuntu-latest to confirm the project compiles with C++11; runs tests and sees if they pass
    - This check is minimal, it confirms compilation and test passing. This is done through Docker, see the Docker section later in this document.
        - Having the test passing confirmation means that on each PR, we are not only ensuring syntactic correctness, but also aiding in catching logical errors much sooner
- check-branch-name: validates the branch name follows prefix/description;
- check-title: validates the PR title follows type: description;
- label: applies the label derived from the branch prefix
    - This, unlike the other three, does not block a merge. The others are gating checks and they enforce the conventions at the point of the merge, whilst this is simply advisory.

- There is a GitHub page associated with the repository which hosts our Doxygen documentation, this is to ensure that the repository is not bloated by the files generated by Doxygen, and there is a clear path to access documentation in a user-friendly manner

Merging

Two merge strategies are permitted:

- Squash and merge is used when the branch contains many small, low-value commits or when the change is a single coherent idea. The pull-request title becomes the single commit message, which is why the title is enforced.
- Merge commit is used when the branch contains commits that each carry meaningful design information and are worth preserving in the history.
    - When a merge commit is used, the default subject line is advisably edited to state what the pull request did, e.g. feat: add WorkItem state hierarchy (#12). This ensures that merge commits are readable in git log and in the GitHub history view.

After merging, the branch is deleted. The pull request remains as a record of the change, including the review conversation, the automated check results and the final approved diff.

Keeping Branches Current

When main advances while a branch is in progress, the author brings the branch up to date before requesting re-review:

```
git checkout main
git pull
git checkout feature/work-item-state
git merge main
git push
```

git rebase main followed by git push --force-with-lease is permitted for branches that only one member is working on, and produces a cleaner history. Rebase is not used on branches that another member is actively working on, and the team is aware that force-pushing dismisses existing approvals.

Overall, this workflow supports the project rules. The design decision log, team contribution statement and revision history will be maintained throughout the project, rather than retrospectively at the end of the project. The log is a genuine record of the design process rather than a summary written after the fact.

Instructions

Launch the TUI with make run or make docker-run. The application presents a numbered menu of installed workflows. Selecting a workflow starts a new instance; the screen prompts for the acting participant, the workflow data, and the item to act on. The two demonstration workflows are:

- transaction-dispute (Banking) — credit card dispute resolution, including an appeal path and a merchant-silence exception.
- service-activation (Telecom) — mobile or fibre service provisioning, including a credit-referral branch and a technician-assignment fork.

make test compiles and runs every tests/*.cpp file. Each test file produces a separate binary under build/tests/ and is run in sequence; the target exits non-zero if any test fails. Under Docker, the same

invocation runs inside the container. After tests are executed, it will show a summary of whether the tests passed or failed.

The test suite covers value objects (Types, Outcome, WorkflowData, Participant, Clock), state transitions, prototype clone isolation, command undo, strategy routing, observer notification, escalation chain resolution, adapter failure handling, and end-to-end execution of both industries.

Docker

Docker can be used with only Docker 20.1+ and make, no other prerequisites, unlike the methods below. The Dockerfile installs the complete toolchain inside the image.

```
# Build the development image (once; ~2–5 minutes on first run, instant after)
make docker-build
# Run 'make' inside the container
make docker-make
# Run 'make test' inside the container (recommended for CI and demo)
make docker-test
# Launch the interactive TUI inside the container
make docker-run
# Remove the image and local build artefacts
make docker-clean
# Same as 'valgrind', inside Docker
make docker-valgrind
# Same as 'valgrind-run', inside Docker
make docker-valgrind-run
```

The container mounts the current directory as /app and runs as the invoking user's UID, so files written to build/ are owned by the developer, not by root. This means make docker-test followed by make clean on the host works without permission errors.

This provides a reproducible development and test environment, motivated by the following reasons:

- Team members run different OS distributions with different installed versions of the compiler, CMake, and test framework. A build that succeeds on one developer's machine has no guarantee of succeeding on another's, and "works on my machine" discrepancies cost team time in debugging environment configuration rather than code.
- GoogleTest dropped C++11 support in version 1.13. A routine system update can silently break the test suite on a machine that previously worked, with a failure that looks like a code error but is actually a framework mismatch. The Dockerfile pins GoogleTest 1.12.1 — the last C++11-compatible release, so this failure mode is eliminated.
- A new contributor (a marker, a recruiter, or a future teammate) can run make docker-test on any host with Docker installed and see the full test suite pass, without configuring a C++11 toolchain or installing a specific GoogleTest version. A new contributor would be able to run the program directly from GitHub's codespaces, since the repository supports Docker-in-Docker, increasing accessibility to the project, anyone can contribute or run the project with zero local setup.

Build, Run & Test

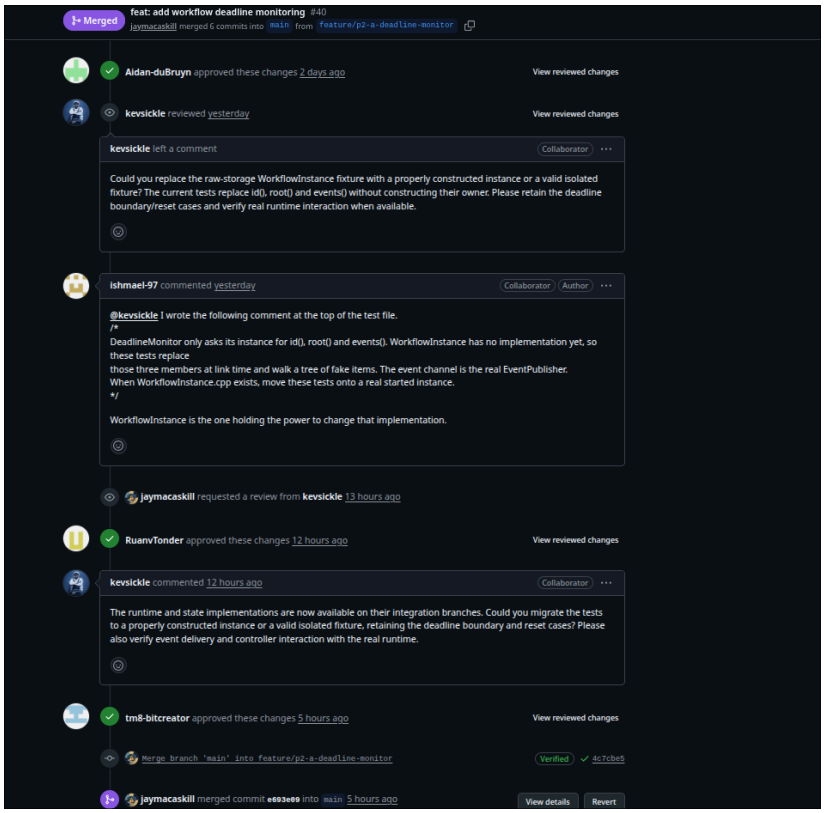
The prerequisites for a local build without Docker are a C++11 compiler (GCC 7+ or Clang 6+), make and GoogleTest 1.12.1 (which is the last release supporting C++11). GoogleTest can be installed system-wide, or omitted if only the local make and make run targets will be used.

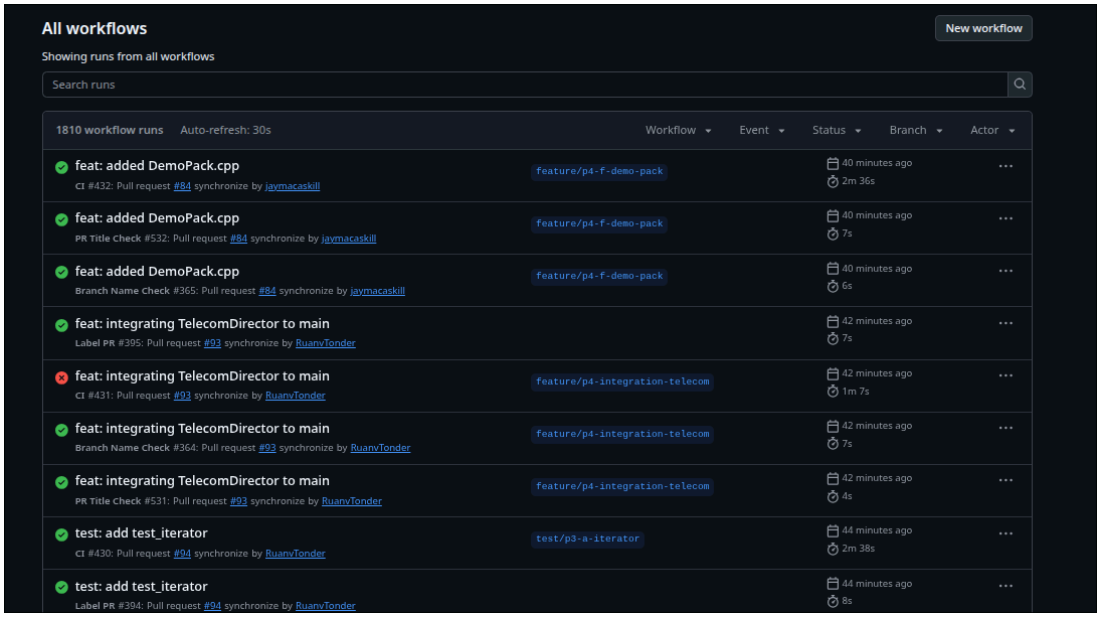
```
# Clone the repository and enter the directory, our project supports make
git clone https://github.com/jaymacaskill/tividy.git
cd tividy
# Build the tividy binary
```

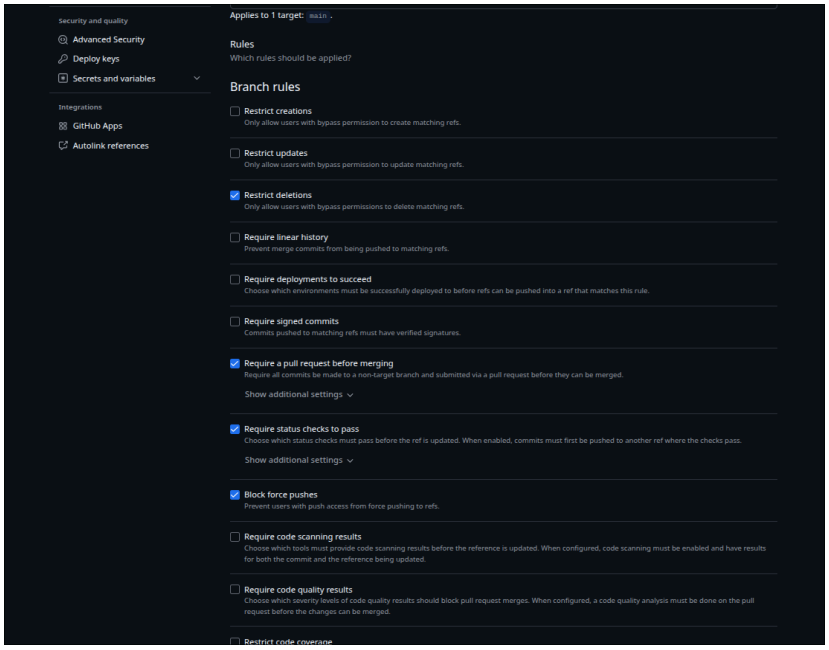
```
make
# Build and immediately run the interactive TUI
make run
# Build and run the test suite automatically, after tests run, there is a clear green “Pass” or
red “Fail”
make test
# Generate Doxygen HTML documentation
make doxygen
# Remove build artefacts
make clean
# Run test suite under valgrind
make valgrind
# Run the TUI under valgrind
make valgrind-run
# List all available targets
make help
```

Git Evidence

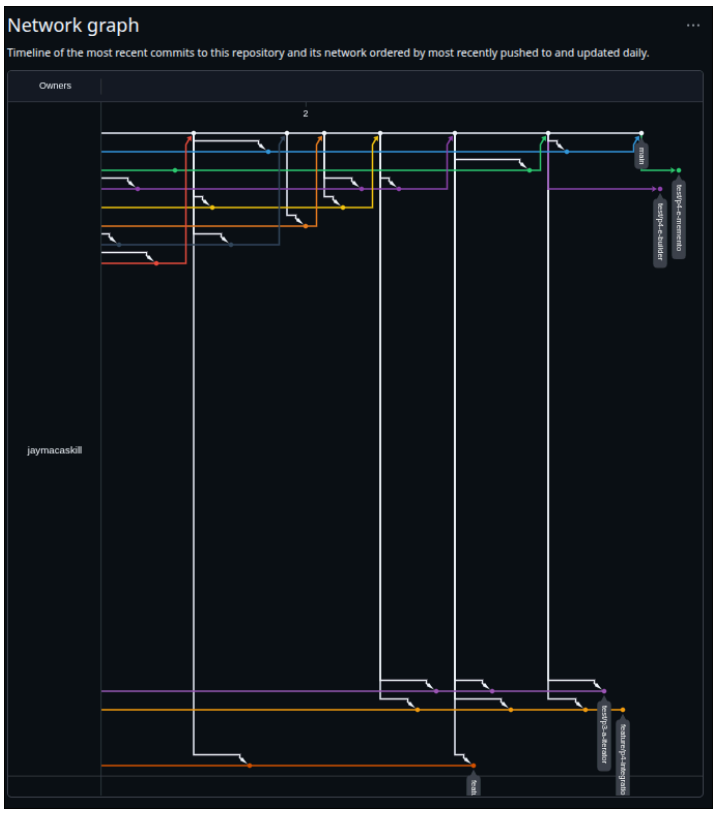





All workflows
New workflow
Showing runs from all workflows
Search runs
1810 workflow runs
Auto-refresh: 30s
Workflow
Event
Status
Branch
Actor
feat: added DemoPack.cpp
CI #432: Pull request #84 synchronize by jaymacaskill
feature/p4-f-demo-pack
40 minutes ago
2m 36s
feat: added DemoPack.cpp
PR Title Check #532: Pull request #84 synchronize by jaymacaskill
feature/p4-f-demo-pack
40 minutes ago
7s
feat: added DemoPack.cpp
Branch Name Check #365: Pull request #84 synchronize by jaymacaskill
feature/p4-f-demo-pack
40 minutes ago
6s
feat: integrating TelecomDirector to main
Label PR #395: Pull request #93 synchronize by RuanvTonder
42 minutes ago
7s
feat: integrating TelecomDirector to main
CI #431: Pull request #93 synchronize by RuanvTonder
feature/p4-integration-telecom
42 minutes ago
1m 7s
feat: integrating TelecomDirector to main
Branch Name Check #364: Pull request #93 synchronize by RuanvTonder
feature/p4-integration-telecom
42 minutes ago
7s
feat: integrating TelecomDirector to main
PR Title Check #531: Pull request #93 synchronize by RuanvTonder
feature/p4-integration-telecom
42 minutes ago
4s
test: add test_iterator
CI #430: Pull request #94 synchronize by RuanvTonder
test/p3-a-iterator
44 minutes ago
2m 38s
test: add test_iterator
Label PR #394: Pull request #94 synchronize by RuanvTonder
44 minutes ago
8s


Security and quality
Advanced Security
Deploy keys
Secrets and variables
Integrations
GitHub Apps
Autolink references
Applies to 1 target: main.
Rules
Which rules should be applied?
Branch rules
Restrict creations
Only allow users with bypass permission to create matching refs.
Restrict updates
Only allow users with bypass permission to update matching refs.
Restrict deletions
Only allow users with bypass permissions to delete matching refs.
Require linear history
Prevent merge commits from being pushed to matching refs.
Require deployments to succeed
Choose which environments must be successfully deployed to before refs can be pushed into a ref that matches this rule.
Require signed commits
Commits pushed to matching refs must have verified signatures.
Require a pull request before merging
Require all commits be made to a non-target branch and submitted via a pull request before they can be merged.
Show additional settings
Require status checks to pass
Choose which status checks must pass before the ref is updated. When enabled, commits must first be pushed to another ref where the checks pass.
Show additional settings
Block force pushes
Prevent users with push access from force pushing to refs.
Require code scanning results
Choose which tools must provide code scanning results before the reference is updated. When configured, code scanning must be enabled and have results for both the commit and the reference being updated.
Require code quality results
Choose which severity levels of code quality results should block pull request merges. When configured, a code quality analysis must be done on the pull request before the changes can be merged.
Restrict code coverage


Weekly from 1 Aug 2026 to 1 Nov 2026
jaymacaskill
Aidan-duBruyn
tm8-bitcreator
RuanvTonder
Conjoss
ishmael-97
kevsickle




```
[       OK ] FacadeStartupTest.RejectsCustomNotifierCollisionWithDefaultNotifier (0 ms)
[ RUN      ] FacadeStartupTest.PreservesSingleCustomNotifierAndRejectsConflictingDefault
[       OK ] FacadeStartupTest.PreservesSingleCustomNotifierAndRejectsConflictingDefault (0 ms)
[ RUN      ] FacadeStartupTest.RegistryStillRejectsDirectDefaultNotifierDuplicates
[       OK ] FacadeStartupTest.RegistryStillRejectsDirectDefaultNotifierDuplicates (0 ms)
[----------] 12 tests from FacadeStartupTest (1 ms total)

[----------] Global test environment tear-down
[==========] 12 tests from 1 test suite ran. (1 ms total)
[  PASSED  ] 12 tests.

--- build/tests/test_progress_visitor_structure ---
Running main() from /tmp/gtest/googletest/src/gtest_main.cc
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from ProgressVisitorStructure
[ RUN      ] ProgressVisitorStructure.StartsWithNoTasksAndZeroCounts
[       OK ] ProgressVisitorStructure.StartsWithNoTasksAndZeroCounts (0 ms)
[----------] 1 test from ProgressVisitorStructure (0 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (0 ms total)
[  PASSED  ] 1 test.

PASS: 50 test file(s) passed
```

